Python 与 PyTorch 基础数据类型及核心知识点

一、整数（int）

- **定义**：整数是正整数、负整数和零的统称，不包含小数和分数部分，是 Python 中最基础的数值类型之一，可用于表示计数、编号等整数场景。
- **示例**：正整数 10、200、1000；负整数-5、-30、-120；零 0，这些均为整数的常见形式。
- **运算**：支持加减乘除（+、-、*、/）基础运算，其中除法运算结果会自动转为浮点数；取模（%）用于计算两个数相除后的余数，如 7%2=1；整除（//）用于获取两个数相除后的整数部分，忽略小数，如 7//2=3。

二、浮点数（float）

- **定义**：浮点数是带小数点的数字，可表示包含小数部分的数值，也可表示科学计数法形式的数（如 1.23e4 即 12300），用于需要精确到小数位的场景。
- **示例**：正数 3.14（圆周率常用值）、0.001（千分之一）；负数-2.5、-0.0001；科学计数法 1.5e3（即 1500）、2.0e-2（即 0.02）。
- **注意**：浮点数运算存在精度误差，这是由于计算机以二进制存储浮点数导致的，例如 0.1+0.2 的结果并非 0.3，而是 0.30000000000000004，实际使用中可通过 round()函数或 decimal 模块修正精度。

三、字符串（str）

- **定义**：字符串是用引号包裹的文本，引号可使用双引号（""）、单引号（''），若文本中包含引号，可使用转义字符（\）或不同类型引号嵌套，是 Python 中用于表示文本信息的核心类型。
- **示例**：双引号包裹的"python"（编程语言名称）、单引号包裹的'hello'（问候语）；嵌套引号示例：'He said "Hello World"'、"I'm a student"；转义字符示例："Hello\nWorld"（换行显示）。
- **操作**：切片用于截取字符串中的部分内容，格式为 str[起始索引:结束索引]，如"python"[1:4]结果为"yth"；拼接可通过+号实现，如"hello"+"world"结果为

"helloworld"；格式化常用 f-string（f"变量：{变量名}"）、format()方法，用于动态插入内容。

四、列表（list）

- **定义**：可修改的序列，用方括号（[]）包裹，内部可存储不同类型的数据（整数、浮点数、字符串等），元素之间用逗号分隔，支持动态添加、删除、修改元素，是 Python 中最常用的容器类型之一。
- **操作**：append()用于在列表末尾添加单个元素，如 list1.append(5)；insert()用于在指定索引位置插入元素，如 list1.insert(1, 10)（在索引 1 处插入 10）；remove()用于删除指定值的元素，如 list1.remove(5)；切片用于截取列表部分元素，用法与字符串切片一致，同时可通过切片修改列表内容，如 list1[1:3] = [20, 30]。

五、字典（dict）

- **定义**：以键值对（key: value）形式存储数据，用大括号（{}）包裹，键（key）需是唯一的、不可变的（如整数、字符串），值（value）可是任意类型数据，用于快速通过键查找对应的值。
- **操作**：获取值可通过 dict[key]（若键不存在会报错）或 dict.get(key)（若键不存在返回 None）；修改值直接通过 dict[key] = 新值实现，如 dict1["name"] = "Tom"；获取所有键可使用 dict.keys()方法，返回包含所有键的可迭代对象，也可通过 for 循环遍历所有键。

六、张量（Tensor）

- **定义**：PyTorch 的基本数据结构，可理解为多维数组，用于存储和处理神经网络中的数据（如输入数据、权重、偏置等），支持 CPU 和 GPU 加速运算，是 PyTorch 中所有操作的核心载体。
- **创建**：torch.tensor()可将 Python 列表、numpy 数组转为张量，如 torch.tensor([1, 2, 3])；torch.zeros()创建全为 0 的张量，如 torch.zeros(2, 3)（创建 2 行 3 列的零张量）；torch.rand()创建 0-1 之间随机值的张量，如 torch.rand(1, 4)（创建 1 行 4 列的随机张量）。
- **操作**：view()用于修改张量的形状（不改变数据），如 tensor.view(3, 2)将张量转为 3 行 2 列；形状变换还可使用 reshape()方法，与 view()功能类似；支持加减乘除、矩阵乘法等运算，如 tensor1 + tensor2、torch.matmul(tensor1, tensor2)（矩阵乘法）。

七、自动求导（autograd）

- **功能**：PyTorch 中的自动求导机制，可自动计算张量的梯度，无需手动推导数学公式，是实现神经网络反向传播、参数更新的核心功能。
- **使用**：创建张量时设置 requires_grad=True，标记该张量需要计算梯度，如 x = torch.tensor(2.0, requires_grad=True)；通过 backward()方法触发梯度计算，如 loss.backward()（对损失函数 loss 求导）；通过 tensor.grad 获取计算出的梯度，如 x.grad 可获取 x 的梯度值。
- **注意**：梯度需要手动清零，因为 PyTorch 中梯度会默认累加，若不手动清零，下次计算的梯度会与之前的梯度叠加，导致参数更新错误，常用 torch.optim.Optimizer.zero_grad()方法清零梯度。

八、数据加载（DataLoader）

- **功能**：PyTorch 中用于批量加载数据的工具，可将自定义数据集或内置数据集（如 MNIST）按照指定批次大小加载，同时支持数据打乱、多线程加载等功能，简化训练过程中的数据迭代操作。
- **参数**：batch_size 用于指定每批加载的数据量，如 batch_size=32 表示每批加载 32 个样本；shuffle 用于设置是否打乱数据顺序，shuffle=True 表示每次迭代时打乱数据，避免模型学习到数据顺序的规律，提升模型泛化能力。
- **用途**：训练时迭代数据，通过 for 循环遍历 DataLoader，每次获取一批数据（输入数据和对应标签），传入模型进行前向传播、损失计算和反向传播，是神经网络训练中不可或缺的工具。